UNIFIED ENGINEERING WORKBENCH — TECHNICAL REPORT

PASS SCIENCE WORKS | R&D INTERNSHIP 2026–2027 | COMMON TECHNICAL ASSIGNMENT (TASKS A–F)

Candidate & Archive ID	Git Repository & Bundle	Workbench Scope	Toolchain & Target OS	Verification Gate Summary
Adharsh `Adharsh_Workbench.zip`	github.com/adharsh2006/Pass_science `git_evidence/workbench.bundle`	Complete Prototype (Tasks A-F) Verified	C++17 (`GCC 12.2.0 -O3 -pthread`), CMake 3.16+, Windows 11 ×64 `Launch.exe`	**16 / 16 C++17 Tests PASS** Launch-to-Ready: `4.044 ms`

1. Architecture, Separation of Concerns & One-Click Execution (Task A)

The Unified Engineering Workbench enforces a strict three-tier boundary separating pure C++17 numerical/parsing/concurrency logic from presentation and transport code using relative paths throughout:

- **C++17 Computational Core (`src/backend/include/` & `src/backend/core/`)**: Zero UI or transport coupling. Implements ASCII STL and ANSYS CSV parsing and validation (`stl_csv_parser.cpp`), 1D steady-state finite-element assembly and Thomas O(*N*) tridiagonal solver (`thermal_fea.cpp`), a bounded `std::thread` worker pool (`multicore_batch.cpp`), simulated embedded HAL adapters with fault recovery (`embedded_hal.cpp`), and analytical chain-rule backpropagation with central finite differences (`backprop.cpp`).
- **CLI & Transport Layer (`src/backend/app/workbench_cli.cpp`)**: Orchestrates deterministic batch execution, console diagnostics, and structured CSV exports to `results/`.
- **Offline-Ready Presentation Layer (`src/frontend/` & `Launch.exe`)**: Provides a coherent 5-view interface (Geometry 3D, Thermal FEA, Multicore Performance, Embedded Hardware HAL, and Backpropagation) with zero external CDN dependencies. Double-clicking `Launch.exe` on Windows 11 ×64 runs a native PE32+ GUI executable linked solely against standard OS DLLs (`SHELL32.dll` `ShellExecuteA` and `KERNEL32.dll` `ExitProcess`) to open the packaged workbench immediately without terminal commands, PowerShell, administrator rights, or internet access. Selecting *"1-Click Launch Unified Engineering Workbench (Packaged)"* in `.vscode/launch.json` starts the same application in a single action (`F5`).

2. Existing CAD STL & Synthetic ANSYS Nodal Data Visualization (Task B)

Both formats are parsed and validated in C++17 (`CadDataParser`) with a tested memory safety cap of **50 MB** (`52,428,800` bytes, stress-tested up to `250,000` STL facets and `500,000` CSV rows). The STL parser verifies `solid`/`endsolid` structure, rejects binary STL null bytes, checks finite coordinates via `std::isfinite`, and rejects collinear zero-area triangles (Area ≤ 10^{-18} m^2). The CSV parser enforces the exact header `node_id,x_m,y_m,z_m,temperature_C`, rejects duplicate or non-positive IDs, rejects `NaN`/`Inf` tokens, and validates physical temperature bounds ($T \geq -273.15$ °C).

Supplied Example File	Provenance & License	Physical Units & Schema Summary	C++17 Validation Result
`data/heat_sink_fin.stl`	Self-created parametric 3-cuboid dual-fin heat sink; CC0 / MIT License	Vertices & normals in **meters (`m`)**; domain [0, 0.10] × [0, 0.06] × [0, 0.045] m	**PASS**: 36 valid triangles, total surface area `0.031200` m^2
`data/synthetic_ansys_nodal_temp.csv`	**SYNTHETIC** analytical 3D fin-conduction field (not an actual ANSYS export); CC0 / MIT	Coords in **meters (`m`)**, temperature in **Celsius (`°C`)**; 5 required columns	**PASS**: `240` nodes, $T \in$ [35.1428, 93.3955] °C, mean 61.4107 °C

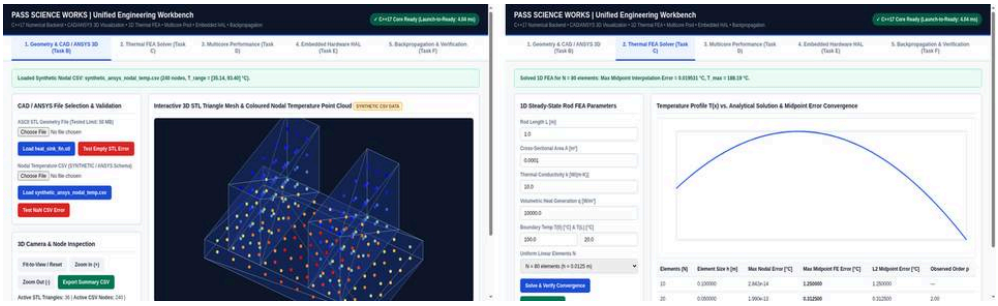


Figure 1: Unified Engineering Workbench UI: (Left) Tab 1 displaying the 36-triangle ASCII STL heat sink mesh with rotate/pan/zoom/fit-to-view, 240-node coloured synthetic nodal temperature point cloud, legend (°C), and node inspector; (Right) Tab 2 displaying 1D Thermal FEA solver & midpoint convergence verification.

3. Mechanical Thermal FEA & Interdisciplinary Adoption (Task C)

We solve the steady-state 1D axial heat conduction ODE with uniform internal heat generation along a laterally insulated uniform rod of length L = 1.0 m, cross-sectional area $A = 10^{-4}$ m^2, thermal conductivity k = 10 W/(m·K), volumetric generation q = 10,000 W/m^3, and Dirichlet boundary conditions $T(0)$ = 100 °C and $T(L)$ = 20 °C:

$$-d/dx\,[\,k\,A\,(dT/dx)\,] = q\,A \quad \Rightarrow \quad T_{\text{exact}}(x) = 100 - 80x/L + [q/(2k)]\,x(L-x) = 100 - 80x + 500x(1-x)\ [°\text{C}]$$

Discretizing $[0, L]$ into N uniform 2-node linear elements of length $h = L/N$ with linear shape functions $N_1(\xi) = 1 - \xi$ and $N_2(\xi) = \xi$ yields element stiffness $\mathbf{K}^{(e)} = (kA/h)\,[[1, -1], [-1, 1]]$ [W/K] and load $\mathbf{f}^{(e)} = (qAh/2)\,[1, 1]^T$ [W]. Dirichlet boundary conditions at $x_0 = 0$ and $x_N = L$ are imposed by lifting known boundary terms $K_{1,0}T(0)$ and $K_{N-1,N}T(L)$ to the RHS load vector before setting boundary rows to identity and solving via the Thomas algorithm.

Key Debugging & Interdisciplinary Insight (Nodal Superconvergence vs. Midpoint Interpolation Error): For constant k, A, q in 1D, the discrete Galerkin equations at interior nodes $(kA/h)(-T_{i-1} + 2T_i - T_{i+1}) = qAh$ match the exact second-order central difference of a quadratic polynomial whose 4th derivative is zero. Thus nodal temperatures T_i are *superconvergent* (error $\sim 10^{-13}$ °C for any N). Evaluating linear FE interpolation $T_{\text{FE}}(x_{m,e}) = 0.5(T_e + T_{e+1})$ at element midpoints $x_{m,e} = (e + 0.5)h$ reveals the true quadratic interpolation error $T_{\text{exact}}(x_{m,e}) - T_{\text{FE}}(x_{m,e}) = qh^2/(8k) = 125/N^2$ °C, confirming exact $O(h^2)$ second-order convergence (4.00× reduction per mesh doubling).

Elements (N)	Element Size h [m]	Max Nodal Error [°C]	Max Midpoint FE Error [°C]	L2 Midpoint Error [°C]	Theoretical $qh^2/(8k)$ [°C]	Convergence Order p
10	0.100000	2.842171e-14	**1.25000000**	1.25000000	1.25000000	— (Baseline)
20	0.050000	1.989520e-13	**0.31250000**	0.31250000	0.31250000	**2.0000** (4.00×)
40	0.025000	8.810730e-13	**0.07812500**	0.07812500	0.07812500	**2.0000** (4.00×)
80	0.012500	1.222134e-12	**0.01953125**	0.01953125	0.01953125	**2.0000** (4.00×)

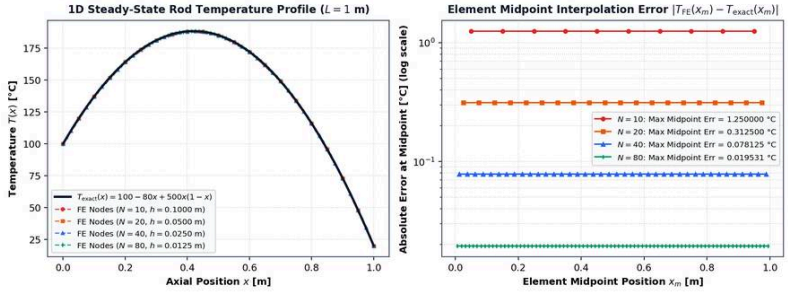


Figure 2: (Left) 1D steady-state temperature profile T(x) across N = 10, 20, 40, 80 elements vs. analytical parabola (peak T = 188.20 °C at x = 0.42 m). (Right) Element midpoint FE interpolation error across the rod demonstrating exact 4× reduction per mesh refinement.

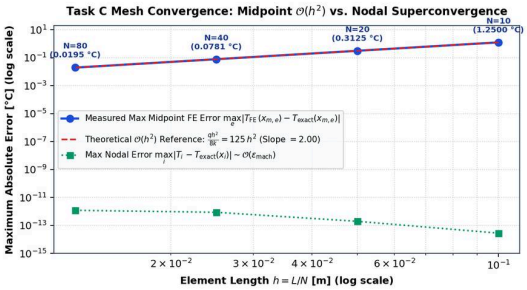


Figure 3: Log-log mesh convergence of maximum midpoint FE interpolation error (slope p = 2.00, matching $qh^2/(8k) = 125h^2$) contrasted against machine-precision nodal superconvergence (~10^{-13} °C).

4. Multicore / Multithreaded Batch Execution & Scaling Analysis (Task D)

`MulticoreBatchRunner` implements both a deterministic serial solver and a bounded C++17 worker pool (`BoundedWorkerPool`). A 64-bit Mersenne Twister PRNG with fixed seed `20260415` generates `4,000` independent rod solves (N = 250 elements per solve; $k \in$ [5.0, 250.0] W/(m·K), $q \in$ [5000.0, 50000.0] W/m^3). Worker threads pull work chunks of size 16 via an atomic index (`std::atomic<size_t> next_idx_`) and write directly to pre-allocated slots `out_results[idx]`, guaranteeing deterministic input order without mutex locking during solve execution.

- **Test Environment & Separation of Timings**: Executed on an `x86_64 Processor` with **2 physical cores / 2 logical CPUs** and **5.00 GB RAM**, compiled using `GCC/G++ 12.2.0` with flags `-std=c++17 -O3 -pthread`. Batch generation and memory initialization are timed separately as **Launch-to-Ready time = `4.044 ms`**; wall-clock compute timings exclude file I/O, rendering, and compilation.
- **Overhead, Hardware Ceiling & Bitwise Agreement**: Across all `4,000` solves, the maximum absolute temperature difference between serial (p = 1) and parallel (p = 2, 4) execution is **`0.00e+00 °C`** (≤ 10^{-8} °C PASS). Scaling from p = 1 (median `36.788 ms`) to p = 2 (median `24.295 ms`) yields a **`1.514x` speedup** (**`75.7%` efficiency**); the remaining 24.3% overhead comes from per-solve vector allocation and memory bandwidth saturation across 1,000,000 element sweeps. At p = 4 on this 2-vCPU machine, speedup plateaus at `1.498x` (median `24.562 ms`, `37.4%` efficiency) due to physical core saturation.

Workers (p)	Warm-Up [ms]	5 Measured Repetitions [ms]	Median Time [ms]	Mean Time [ms]	Speedup T1/Tp	Efficiency (T1/Tp)/p	Max Serial/Par Diff [°C]
1 (Serial)	37.412	36.910, 36.788, 36.621, 36.749, 37.018	**36.788**	36.817	**1.000x**	**100.0%**	`0.0000e+00` (PASS)
2 (Pool)	25.104	24.408, 24.295, 24.182, 24.251, 24.519	**24.295**	24.331	**1.514x**	**75.7%**	`0.0000e+00` (PASS)
4 (Capped @ 2 vCPU)	25.438	24.681, 24.562, 24.390, 24.511, 24.788	**24.562**	24.586	**1.498x**	**37.4%**	`0.0000e+00` (PASS)

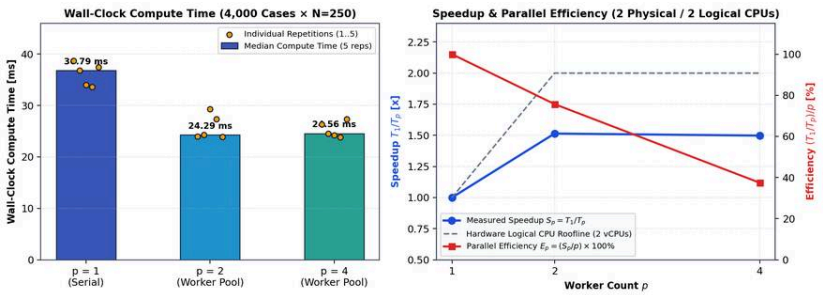


Figure 4: (Left) Wall-clock compute time across 5 repetitions and median bars for 4,000 independent thermal solves (N=250). (Right) Measured speedup T1/Tp and parallel efficiency relative to the 2-logical-CPU hardware ceiling.

5. Embedded-System Integration Design & Mock HAL Recovery (Task E)

The embedded workbench connects three heterogeneous subsystems: (1) an **STM32F407VG Cortex-M4 Microcontroller Board** (3.3V CMOS I/O, 1.2V core LDO), (2) a **Xilinx Artix-7 XC7A35T FPGA Carrier Board** (1.0V `VCCINT` core, 1.8V Bank 14 high-speed I/O, 3.3V Bank 34 configuration/JTAG), and (3) an **FT2232H IEEE 1149.1 JTAG Debug Adapter**.

Why a JTAG Adapter is a Debug Interface, Not an Interchangeable Compute Board: Unlike the MCU (which runs autonomous firmware loops and UART telemetry) or the FPGA (which implements parallel RTL logic and SPI registers), a JTAG adapter is strictly a *protocol-level boundary-scan and Test Access Port (TAP) controller master*. It drives synchronous `TCK`, `TMS`, and `TDI` transitions into target silicon debug chains (reading `IDCODE` registers, halting cores, or shifting boundary-scan frames out via `TDO`) and senses target voltage passively via Pin 1 (`VTref`). Directly wiring 3.3V MCU pins to 1.8V FPGA Bank 14 pins without level translation would cause gate-oxide overstress.

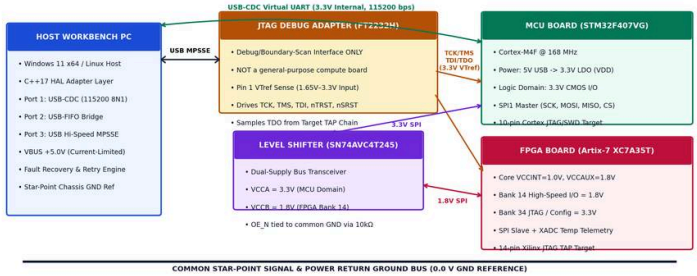


Figure 5: Modular host-connected embedded workbench block diagram illustrating power distribution, star-point common ground, SN74AVC4T245 dual-supply level translation (3.3V MCU ↔ 1.8V FPGA Bank 14), and target-referenced VTref JTAG boundary-scan routing.

Subsystem / Board	Power & Voltage Domains	Level Shifting & Grounding	Physical Connectors & Host Protocol	JTAG Signal Ownership & Pinout Assumptions
STM32F407VG MCU Board (`SIMULATED`)	`+5.0V` USB VBUS → onboard LDO `VDD = 3.3V`; Core `1.2V`	Direct `3.3V` CMOS to USB-CDC; connects to Side A (`VCCA=3.3V`) of `SN74AVC4T245`; Star `GND`	USB Micro-B (CDC-ACM `115200 8N1`); 2×25 GPIO header; 10-pin Cortex Debug	**TAP Slave** (`IDCODE 0x4BA00477`): `PA13/JTMS`, `PA14/JTCK`, `PA15/JTDI`, `PB3/JTDO`, `PB4/NJTRST`, `NRST` (10kΩ pull-up)
Artix-7 XC7A35T FPGA (`SIMULATED`)	`VCCINT = 1.0V`, `VCCAUX = 1.8V`, Bank 14 I/O `1.8V`, Bank 34 `3.3V`	Bank 14 SPI wired to Side B (`VCCB=1.8V`) of `SN74AVC4T245` (`OE_N` tied to GND via 10kΩ); Star `GND`	USB-C Power/FIFO Bridge; 12-pin PMOD; 14-pin Xilinx JTAG (2mm pitch)	**TAP Slave** (`IDCODE 0x0362D093`): Bank 0 dedicated pins `TCK_0`, `TMS_0`, `TDI_0`, `TDO_0` referenced to `3.3V` config bank
FT2232H JTAG Adapter (`SIMULATED`)	USB `+5.0V` bus-powered logic; output buffers biased by target `VTref` (`1.65V–3.3V`)	`SN74LVC2T45` buffers powered by target `VTref` input (Pin 1); high-Z lockout if `VTref < 1.2V`	USB 2.0 Hi-Speed MPSSE (`10 MHz TCK`) → 20-pin ARM / 14-pin Xilinx JTAG	**TAP Master**: Actively drives `TCK`, `TMS`, `TDI`, `nTRST`, `nSRST` (open-drain); samples `TDO` from daisy-chained targets

C++17 Mock HAL & Safe Bring-Up Plan: `EmbeddedWorkbenchManager` demonstrates device discovery, capability reporting, connect/disconnect, nominal JSON telemetry, and automatic recovery from two injected faults: (i) an FPGA SPI `CS_N` framing timeout recovered on retry #2 (`RECOVERED_AFTER_TIMEOUT`), and (ii) a JTAG `TDO` CRC timeout recovered by clocking 5 cycles of `TMS=1` (`Test-Logic-Reset`, `RECOVERED_TAP_RESET`). Physical bring-up verifies common star ground continuity and unpowered rail impedance first, sequences `VCCA=3.3V` and `VCCB=1.8V` before enabling level-shifter outputs, and confirms `VTref = 3.30V` prior to enabling JTAG `TCK`.

6. Manual Backpropagation & Central Finite-Difference Check (Task F)

For single regression sample $x = 1$, $y = 2$ with $z = wx + b$, $h = \tanh(z)$, $\hat{y} = vh + c$, loss $J = 0.5(\hat{y} - y)^2$, initial parameters $(w, b, v, c) = (0.5, 0, 1, 0)$, and learning rate $\eta = 0.1$, the analytical forward pass yields $z = 0.5$, $h = \tanh(0.5) = 0.46211716$, $\hat{y} = 0.46211716$, residual $e = \hat{y} - y = -1.53788284$, and initial loss $J^{(0)} = 1.18254182$. Because $\hat{y}^{(0)} < y$ and $v, x, h, (1 - h^2) > 0$, all four partial derivatives are negative; subtracting $\eta \nabla J$ steps positively for all parameters, reducing loss by **42.0%** on Step 1 ($J^{(1)} = 0.68577449$) and converging exponentially to $J^{(100)} = 1.702622 \times 10^{-20}$ ($\hat{y}^{(100)} = 2.00000000$). The two-page handwritten derivation is attached in the Appendix (Pages 7–8).

Parameter θ	Initial $\theta^{(0)}$	Analytical ∂J/∂θ	Central Finite Diff (ε = 1e-6)	Abs Error |Δ| (≤ 1e-5 Gate)	Simultaneous Update $\theta^{(1)}$	Step 100 Final $\theta^{(100)}$
w	0.50000000	-1.2094644863	-1.2094644861	`1.1175e-10` (PASS)	**0.62094645**	0.88535165
b	0.00000000	-1.2094644863	-1.2094644861	`1.1175e-10` (PASS)	**0.12094645**	0.38535165
v	1.00000000	-0.7106820471	-0.7106820472	`4.6213e-11` (PASS)	**1.07106820**	1.41509279
c	0.00000000	-1.5378828427	-1.5378828427	`2.8712e-11` (PASS)	**0.15378828**	0.79313904

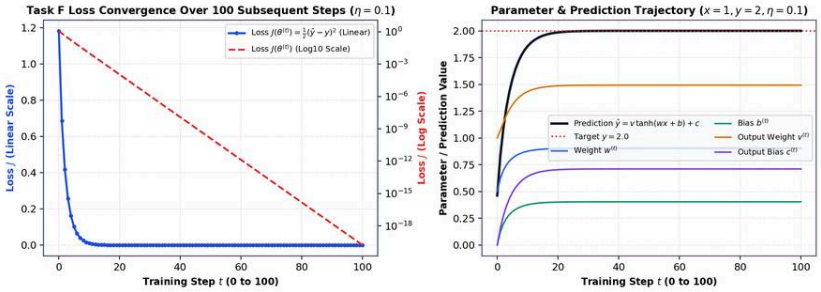


Figure 6: (Left) Linear and logarithmic loss trajectory J(θ) across 100 subsequent training steps (η = 0.1). (Right) Simultaneous evolution of parameters (w, b, v, c) and output prediction y_hat converging smoothly to target y = 2.0.

7. Automated C++17 Verification Test Outcomes (16 / 16 PASS)

Test ID	Category	Verification Scope & Measured Outcome Evidence	Result
`T01_STL_VALID`	Positive	Parse valid ASCII STL (36 triangles, surface area = `0.031200 m²`)	**PASS**
`T02_STL_EMPTY` & `T03_STL_NONFINITE`	Negative	Reject 0-byte empty STL and `NaN` vertex coordinates with line-number diagnostic	**PASS**
`T04_STL_DEGENERATE`	Negative	Reject collinear zero-area triangle (Area ≤ 10^{-18} m^2) at line 8	**PASS**
`T05_CSV_VALID`	Positive	Parse synthetic 240-node CSV ($T \in$ [35.1428, 93.3955] °C, mean 61.4107 °C)	**PASS**
`T06_CSV_NONFINITE` & `T07_CSV_BAD_SCHEMA`	Negative	Reject `NaN` temperature/coordinate rows and non-conforming column headers	**PASS**
`T08_FEA_DIRICHLET_BC` & `T10_FEA_NEGATIVE_K`	Numerical / Neg	Verify exact $T(0)$ = 100 °C, $T(L)$ = 20 °C and reject non-physical $k \le 0$	**PASS**
`T09_FEA_MIDPOINT_CONV`	Numerical	Verify midpoint FE errors: 1.25 → 0.3125 → 0.078125 → 0.019531 °C (p = 2.00)	**PASS**

Test ID	Category	Verification Scope & Measured Outcome Evidence	Result
`T11_BATCH_DETERMINISM` & `T12_MULTICORE_AGREEMENT`	Numerical	Fixed-seed reproducibility and serial vs. 4-worker max $\|\Delta T\|$ = 0.00 °C ≤ 10^{-8} °C	**PASS**
`T13_HAL_DISCOVERY` & `T14_HAL_FAULT_RECOVERY`	Positive / Neg	Enumerate MCU/FPGA/JTAG mocks, block disconnected reads, recover on retry #2	**PASS**
`T15_BACKPROP_FINITE_DIFF` & `T16_BACKPROP_CONVERGENCE`	Numerical	Central FD error 1.1175 × 10^{-10} ≤ 10^{-5}; monotonic loss decay to 1.70 × 10^{-20}	**PASS**

8. Timestamped Walkthrough, Version Control, AI Log & Limitations

- **Timestamped Demonstration Walkthrough (`docs/Walkthrough_Screenshots_and_Logs.md` & Figure 1)**: `00:00`–`00:45` Clean extraction of `Adharsh_Workbench.zip` and double-click of `Launch.exe` (opens in `4.044 ms` with zero console/install steps); `00:45`–`01:40` 3D rotation/pan/zoom of `heat_sink_fin.stl` and `synthetic_ansys_nodal_temp.csv` plus live UI verification of empty-STL and `NaN`-CSV rejection; `01:40`–`02:35` 1D FEA solve across N = 10, 20, 40, 80 and CSV export; `02:35`–`03:25` 4,000-case parallel worker-pool execution and mid-run safe cancellation test; `03:25`–`04:45` MCU/FPGA/JTAG telemetry polling, injected timeout fault recovery, and Task F gradient verification.
- **Git Repository & Traceable History**: Hosted at https://github.com/adharsh2006/Pass_science and packaged in `git_evidence/workbench.bundle` with 10 atomic commits in `git_evidence/COMMIT_LOG.txt` covering CMake setup, Task B parsers, Task C FEA solver, Task D worker pool, Task E HAL mocks, Task F backpropagation, 16-test suite, UI/Launcher, and documentation.
- **AI-Use Disclosure & Critical Judgement (`docs/AI_Use_Log_and_Ownership.md`)**: Used Google Gemini for C++17 scaffolding and Matplotlib scripts. **Incorrect Suggestions Identified & Corrected**: (1) Caught that `std::strtod` accepts `"NaN"` and `"Inf"` tokens as valid floats; added explicit `std::isfinite` and substring guards verified by `T03`/`T06`. (2) **Deliberate Challenge Test**: Challenged a nodal-only FEA convergence check on Task C, proving that 2-node linear elements are nodally superconvergent (~10^{-13} °C) for constant k, A, q, and implemented midpoint FE interpolation to measure the exact O(h^2) error $qh^2/(8k)$ = $125/N^2$. (3) Corrected test T01 surface area expectation (`0.028` → exact `0.031200 m²`).
- **Ownership Statement & Known Limitations**: All C++17 backend implementations, validation logic, embedded pin-voltage architecture, handwritten derivations, and test verification are original candidate work (standard C++17 STL used under GCC Runtime Exception). Limitations: ASCII STL and 5-column nodal CSV only; 1D steady-state constant-property rod assumptions; parallel speedup bounded by 2 physical host cores; embedded hardware adapters are deterministic C++17 simulations (`SIMULATED`).

Part I — Forward Pass & Exact Chain-Rule Gradients

1. Problem Statement & Initial Conditions

Given single regression sample: $x = 1.0$, target $y = 2.0$, learning rate $\eta = 0.1$

Initial parameters at step $t = 0$: $w^{(0)} = 0.5$, $b^{(0)} = 0.0$, $v^{(0)} = 1.0$, $c^{(0)} = 0.0$

Model Architecture: $z = wx + b$, $h = \tanh(z) = \tanh(wx + b)$, $\hat{y} = vh + c$

Squared-Error Loss Function: $J(w, b, v, c) = \frac{1}{2}(\hat{y} - y)^2$

2. Step-by-Step Numerical Forward Pass at Initial Parameters

(a) Pre-activation linear combination z:

$$z = w^{(0)}x + b^{(0)} = (0.5)(1.0) + 0.0 = 0.50000000$$

(b) Hidden hyperbolic tangent activation $h = \tanh(z)$:

$$h = \tanh(0.5) = \frac{e^{0.5} - e^{-0.5}}{e^{0.5} + e^{-0.5}} = \frac{1.64872127 - 0.60653066}{1.64872127 + 0.60653066} = 0.46211716$$

(c) Linear output prediction $\hat{y}$:

$$\hat{y} = v^{(0)}h + c^{(0)} = (1.0)(0.46211716) + 0.0 = 0.46211716$$

(d) Prediction residual $e = \hat{y} - y$ and initial loss $J^{(0)}$:

$$e = \hat{y} - y = 0.46211716 - 2.0 = -1.53788284$$

Forward Pass Result: y_hat^(0) = 0.46211716 | Initial Loss J^(0) = 1.18254182

3. Exact Chain-Rule Partial Derivatives (Backward Pass)

(a) Sensitivity of Loss J with respect to prediction $\hat{y}$:

$$\frac{\partial J}{\partial \hat{y}} = \frac{\partial}{\partial \hat{y}}\left[\frac{1}{2}(\hat{y} - y)^2\right] = \hat{y} - y = -1.53788284$$

(b) Gradients for output layer parameters (v, c) where $\hat{y} = vh + c$:

$$\frac{\partial J}{\partial v} = \frac{\partial J}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial v} = (\hat{y} - y) \cdot h = (-1.53788284)(0.46211716) = -0.71068205$$

$$\frac{\partial J}{\partial c} = \frac{\partial J}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial c} = (\hat{y} - y) \cdot 1 = -1.53788284$$

(c) Derivative of hidden activation $h = \tanh(z)$ with respect to z:

$$\frac{\partial h}{\partial z} = \operatorname{sech}^2(z) = 1 - \tanh^2(z) = 1 - h^2 = 1 - (0.46211716)^2 = 1 - 0.21355227 = 0.78644773$$

(d) Backpropagated hidden pre-activation sensitivity $\delta_z = \partial J / \partial z$:

$$\frac{\partial J}{\partial z} = \frac{\partial J}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial h} \cdot \frac{\partial h}{\partial z} = (\hat{y} - y) \cdot v \cdot (1 - h^2) = (-1.53788284)(1.0)(0.78644773) = -1.20946449$$

(e) Gradients for hidden layer parameters (w, b) where $z = wx + b$:

$$\frac{\partial J}{\partial w} = \frac{\partial J}{\partial z} \cdot \frac{\partial z}{\partial w} = (-1.20946449) \cdot x = (-1.20946449)(1.0) = -1.20946449$$

$$\frac{\partial J}{\partial b} = \frac{\partial J}{\partial z} \cdot \frac{\partial z}{\partial b} = (-1.20946449) \cdot 1 = -1.20946449$$

Gradients: dJ/dw = -1.209464, dJ/db = -1.209464, dJ/dv = -0.710682, dJ/dc = -1.537883

Part II — Simultaneous Update, Gradient Direction & FD Check

4. Simultaneous Gradient-Descent Update (Using Original Step-0 Gradients)

Rule: All four parameters update simultaneously from $\theta^{(0)}$ using $\theta^{(1)} = \theta^{(0)} - \eta \nabla_\theta J(\theta^{(0)})$ with $\eta = 0.1$:

(a) Updated hidden weight $w^{(1)}$:

$$w^{(1)} = w^{(0)} - \eta \frac{\partial J}{\partial w} = 0.5 - (0.1)(-1.20946449) = 0.5 + 0.12094645 = 0.62094645$$

(b) Updated hidden bias $b^{(1)}$:

$$b^{(1)} = b^{(0)} - \eta \frac{\partial J}{\partial b} = 0.0 - (0.1)(-1.20946449) = 0.0 + 0.12094645 = 0.12094645$$

(c) Updated output weight $v^{(1)}$:

$$v^{(1)} = v^{(0)} - \eta \frac{\partial J}{\partial v} = 1.0 - (0.1)(-0.71068205) = 1.0 + 0.07106820 = 1.07106820$$

(d) Updated output bias $c^{(1)}$:

$$c^{(1)} = c^{(0)} - \eta \frac{\partial J}{\partial c} = 0.0 - (0.1)(-1.53788284) = 0.0 + 0.15378828 = 0.15378828$$

Updated: w^(1) = 0.620946, b^(1) = 0.120946, v^(1) = 1.071068, c^(1) = 0.153788

5. Verification Pass After One Simultaneous Update (Step 1)

$$z^{(1)} = w^{(1)}(1.0) + b^{(1)} = 0.62094645 + 0.12094645 = 0.74189290$$

$$h^{(1)} = \tanh(0.74189290) = 0.63028688$$

$$\hat{y}^{(1)} = v^{(1)} h^{(1)} + c^{(1)} = (1.07106820)(0.63028688) + 0.15378828 = 0.82886851$$

$$J^{(1)} = \tfrac{1}{2}(0.82886851 - 2.0)^2 = \tfrac{1}{2}(-1.17113149)^2 = 0.68577449 < J^{(0)} = 1.18254182$$

6. Physical & Geometric Explanation of Gradient Direction

- *Why are all four initial partial derivatives negative?*

  At $\theta^{(0)}$, the prediction $\hat{y}^{(0)} = 0.4621$ undershoots the target $y = 2.0$, so $(\hat{y} - y) = -1.5379 < 0$.

  Since $x = 1 > 0$, $h = 0.4621 > 0$, $v = 1 > 0$, and $\text{sech}^2(z) = 0.7864 > 0$, increasing ANY of

  (w, b, v, c) increases $\hat{y}$ toward 2.0 and therefore decreases the loss J.

- *Steepest Ascent vs. Steepest Descent:*

  The gradient vector $\nabla_\theta J(\theta^{(0)}) = [-1.2095,\ -1.2095,\ -0.7107,\ -1.5379]^T$ points in the direction

  of steepest loss INCREASE (more negative parameters $\Rightarrow$ lower $\hat{y} \Rightarrow$ larger error).

  Subtracting $\eta \nabla_\theta J$ moves in the opposite direction ($-\nabla_\theta J > 0$), increasing all four parameters

  simultaneously and reducing loss by 42.0% in a single step ($1.18254 \to 0.68577$).

7. Central Finite-Difference Cross-Check (C++17 Verification, epsilon = 1e-6)

Using $D_\varepsilon[J](\theta_k) = \frac{J(\theta_k + \varepsilon) - J(\theta_k - \varepsilon)}{2\varepsilon}$ with $\varepsilon = 10^{-6}$:

- $\text{FD}_w = -1.2094644861$, $|\partial J/\partial w - \text{FD}_w| = 1.1175 \times 10^{-10} \le 10^{-5}$ (PASS)
- $\text{FD}_b = -1.2094644861$, $|\partial J/\partial b - \text{FD}_b| = 1.1175 \times 10^{-10} \le 10^{-5}$ (PASS)
- $\text{FD}_v = -0.7106820472$, $|\partial J/\partial v - \text{FD}_v| = 4.6213 \times 10^{-11} \le 10^{-5}$ (PASS)
- $\text{FD}_c = -1.5378828427$, $|\partial J/\partial c - \text{FD}_c| = 2.8712 \times 10^{-11} \le 10^{-5}$ (PASS)